STUDENT TIMETABLE CONFLICT DETECTION

Case Study Report

This case study uses Python to check a student's timetable for overlapping class times and room conflicts. It compares course pairs and reports whether the timetable is valid.

1. Problem Statement

Create a Python program that checks a timetable containing subjects, start times, end times, and room names. The program must detect overlapping class periods and identify cases where the same room is assigned to two classes at the same time. Finally, it should display whether the timetable is valid or contains conflicts.

2. Objective

The objective is to understand how lists, nested loops, tuple unpacking, comparisons, and conditional statements can be used to validate a timetable and detect scheduling problems.

3. Assumptions

• Each timetable entry contains a subject name, start time, end time, and room name.

• The sample timetable includes Maths (9–10, Room1), Python (10–11, Room2), and English (11–12, Room1).

• Start and end times are represented as whole-number hours.

• A conflict occurs when two class time intervals overlap.

• Two classes using the same room at overlapping times cause a room conflict.

• Classes that meet exactly at an end/start boundary, such as 9–10 and 10–11, are not considered overlapping.

4. Algorithm

1. Start the program.

2. Create a schedule list with the subject, start time, end time, and room for each class.

3. Initialize the variable valid to True.

4. Use nested loops to compare each course with every course after it.

5. Check whether the two time intervals overlap.

6. If they overlap, print a time-conflict message and set valid to False.

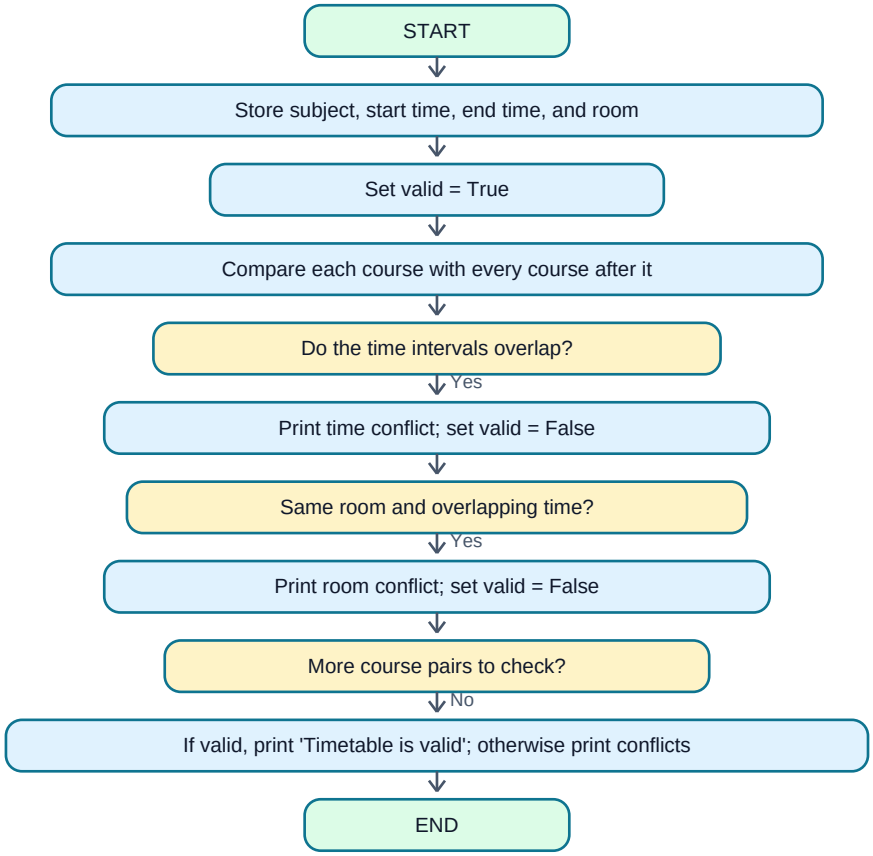
7. If the room names are the same and the times overlap, print a room-conflict message and set valid to False.

8. After checking all pairs, print that the timetable is valid if valid is True; otherwise, print that the timetable has conflicts.

9. Stop the program.

5. Flowchart

The flowchart summarizes the timetable validation process, including pairwise time checks and the final validity message.



6. Pseudocode

```
BEGIN
    schedule ← list of (subject, start, end, room)
    valid ← True

    FOR i ← 0 TO length(schedule) - 1
        FOR j ← i + 1 TO length(schedule) - 1
            Read course1, start1, end1, room1
            Read course2, start2, end2, room2

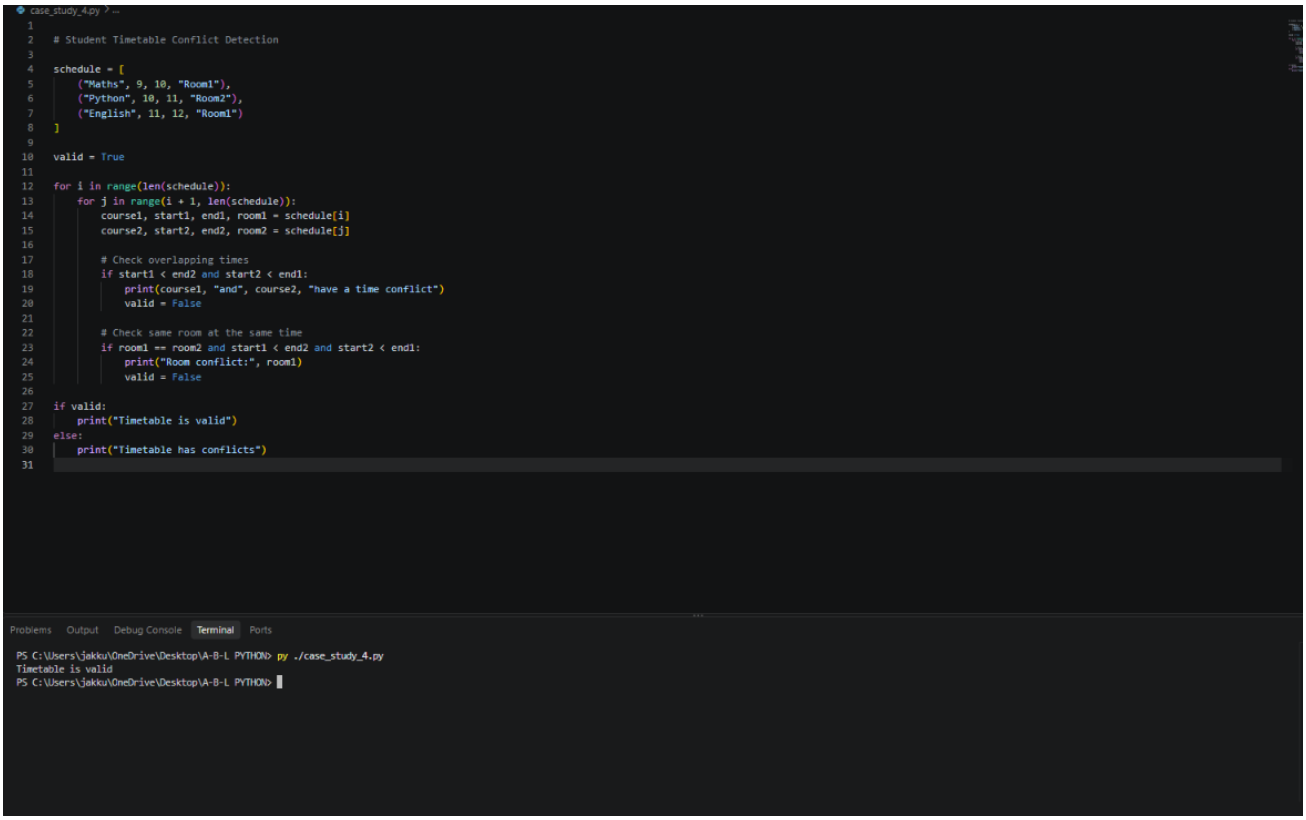
            IF start1 < end2 AND start2 < end1 THEN
                PRINT course1, course2, "have a time conflict"
                valid ← False
            END IF

            IF room1 = room2 AND start1 < end2 AND start2 < end1 THEN
                PRINT "Room conflict:", room1
                valid ← False
            END IF
        END FOR
    END FOR

    IF valid = True THEN
        PRINT "Timetable is valid"
    ELSE
        PRINT "Timetable has conflicts"
    END IF
END
```

7. Screenshot of Python Source Code and Output

The screenshot below shows the original Python source code and the terminal output. For the provided sample schedule, the output is “Timetable is valid.”



8. Conclusion

The Student Timetable Conflict Detection program checks class times and room assignments by comparing pairs of courses. In the supplied example, the classes occupy consecutive non-overlapping time slots, so the program displays “Timetable is valid.” This case study demonstrates how Python can help identify timetable conflicts and support better class scheduling.